



kubernetes

KUBERNETES NOTES

Beginner → Advanced Study Guide

Mohamed Ibrahim

DevOps / Cloud Engineering

Version 1.0
March 2026

Kubernetes

Table of Contents

1	Introduction to Kubernetes	4
2	Core Concepts	4
3	Kubernetes Architecture	7
4	Pod Lifecycle	10
5	kubectl	11
6	Container Runtime & CRI Tools	13
7	Pods	15
8	Deployments	17
9	YAML Basics	20
10	Kubernetes YAML Basics	24
1 1	Replication in Kubernetes	28
1 2	Deployments in Kubernetes	37
1 3	Deployment Strategies and Updates	41
1 4	Rollout and Rollback in Kubernetes	49
1 5	Kubernetes Networking	52
1 6	Kubernetes Services	56
1 8	Port Mapping in Kubernetes	60
2 2	Microservices Architecture Example (Voting App)	67
2 3	Deployments in Microservices Architecture	74

1 Introduction to Kubernetes

What is Kubernetes?

Kubernetes (abbreviated as K8s) is an open-source system used to manage and run applications that operate inside Containers.

What does Kubernetes do?

- Runs applications
- Distributes them across servers
- Performs Scaling (increase/decrease based on load)
- Restarts applications if they fail
- Provides Load Balancing
- Manages networking and storage

Summary

Kubernetes is a container management system designed for Production environments.

2 Core Concepts

2.1 What is a Container?

A Container is a method for running an application along with all its dependencies in an isolated environment.

Instead of running the application directly on a server, it runs inside a Container.

The most popular tool for creating Containers:

- Docker
-

2.2 What is a Cluster?

A Cluster is a group of servers (Machines or Nodes) connected together and working as one system.

2 Core Concepts

Instead of using a single server to run an application, multiple servers share the workload.

A Kubernetes Cluster consists of:

- Control Plane
 - Worker Nodes
-

2.3 What is a Node?

A Node "Minion" is a server (physical or virtual machine) inside a Cluster.

There are two types:

1. Control Plane Node
Responsible for management and decision-making.
 2. Worker Node
Responsible for running Pods and Containers.
-

2.4 What is a Pod?

A Pod is the smallest deployable unit in Kubernetes.

It consists of:

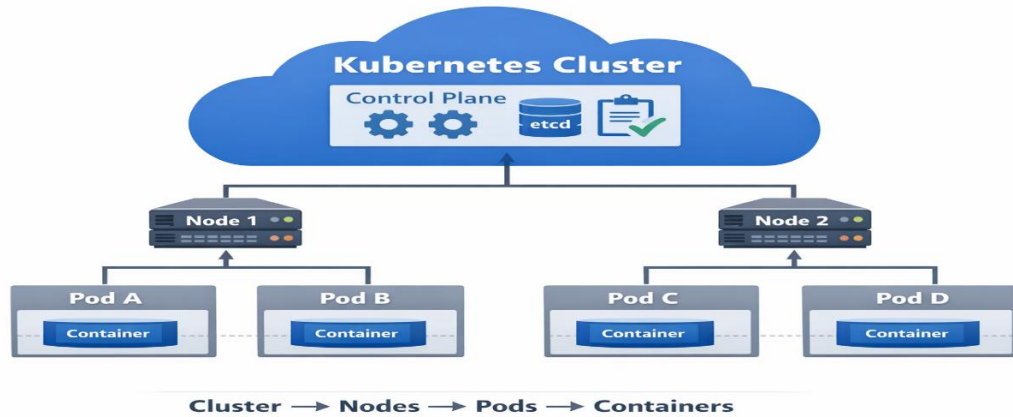
- One or more Containers
- Shared network
- A single shared IP address

Important:

Kubernetes does not run Containers directly.

It runs Pods that contain Containers.

2 Core Concepts



2.5 What is a Deployment?

A Deployment is a way to manage Pods.

It allows you to:

- Define the number of Replicas
- Perform Rolling Updates
- Roll back to a previous version

2.6 What is a Replica?

A Replica means a copy of your application.

If you request 3 Replicas,
Kubernetes will run 3 Pods of the same application.

If one fails → it automatically replaces it.

2.7 What is a Service?

A Service is a way to expose an application inside or outside the Cluster.

Because Pods:

- Can be recreated
- Can change IP addresses

3 Kubernetes Architecture

A Service provides:

- A stable IP address
 - Load Balancing
-

2.8 What is Scaling?

Scaling means increasing or decreasing the number of Pods based on workload.

There are two types:

- Manual Scaling
 - Auto Scaling (such as HPA)
-

2.9 What is Desired State?

Desired State is a core concept in Kubernetes.

You define the state you want.

For example:

"I want 4 Pods running."

Kubernetes ensures that 4 Pods are always running.

If one crashes → it replaces it automatically.

3 Kubernetes Architecture

Kubernetes is divided into two main parts:

1. Control Plane
 2. Worker Nodes
-

3.1 Control Plane Components

API Server (kube-apiserver)

3 Kubernetes Architecture

Function:

- The official entry point of the Cluster
- Receives requests from:
 - kubectl
 - Dashboard
 - CI/CD tools
- Authenticates and authorizes requests
- Stores data in etcd
- Communicates with other components

Summary:

It is the official gateway of the Cluster.

ETCD

Function:

- Stores all Cluster data
- Pods
- Deployments
- Services
- Secrets
- Node Information

Important Note:

If etcd stops → the entire Cluster is affected.

Summary:

The Cluster database (Key-Value Store).

Controller Manager

3 Kubernetes Architecture

Function:

Maintains the Desired State.

Examples:

- Node Controller
- ReplicaSet Controller
- Deployment Controller

Summary:

It corrects any difference between the current state and the desired state.

Scheduler

Function:

Determines which Node will run a Pod.

It compares Nodes based on:

- CPU
- RAM
- Available resources
- Affinity
- Taints & Tolerations

Summary:

It distributes workloads across servers.

3.2 Worker Node Components

Kubelet

An agent that runs on every Node.

Function:

Pod Lifecycle

- Runs Pods
 - Monitors them
 - Reports status to the Control Plane
-

Kube Proxy

Responsible for networking inside the Cluster.

Function:

- Service Routing
 - Internal Load Balancing
-

Container Runtime

Responsible for actually running Containers.

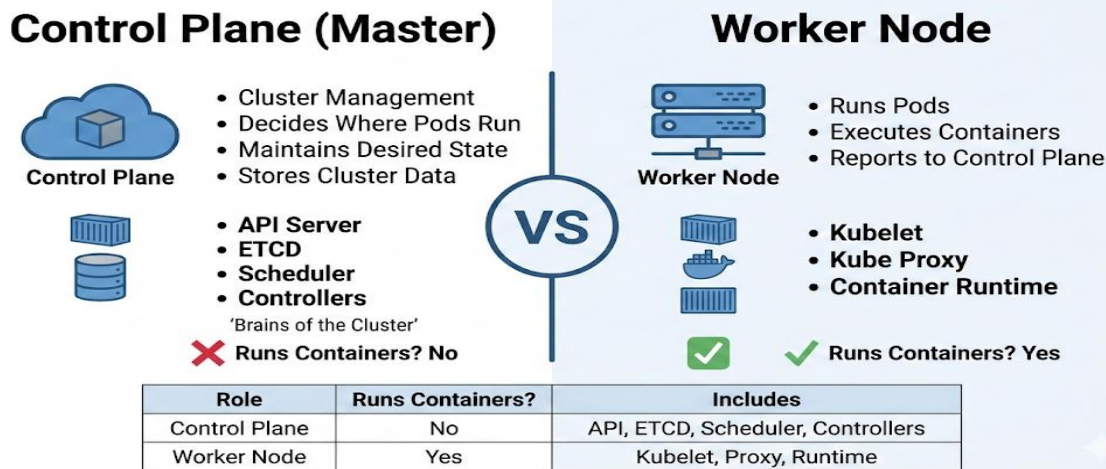
Examples:

- containerd
 - CRI-O
-

Pod Lifecycle

1. Execute a kubectl command
2. The request reaches the API Server
3. The desired state is stored in etcd
4. The Scheduler selects a Node
5. The Kubelet runs the Pod
6. Controllers maintain the state

5 kubectl



5 kubectl

What is kubectl?

kubectl is the official Command Line Tool used to interact with a Kubernetes Cluster.

In simple terms:

It is the tool that allows you to communicate with the API Server and control the Cluster from the terminal.

It acts as a Client,
while the API Server acts as the Server.

How does kubectl work?

1. You type a command in the terminal.
2. kubectl sends the request to the API Server.
3. The API Server validates the request.
4. The change is applied inside the Cluster.
5. The result is returned to your terminal.

kubectl does not execute anything directly.
It only sends requests to the Control Plane.

Common kubectl Commands

View Resources

- kubectl get pods
- kubectl get nodes
- kubectl get services
- kubectl get deployments

These commands display the existing resources inside the Cluster.

View Detailed Information

- kubectl describe pod pod-name

This shows:

- Events
- IP address
- Errors
- Crash reasons

It is very important for troubleshooting.

Apply a YAML File

- kubectl apply -f deployment.yaml

This applies a YAML configuration file to the Cluster.

This is the most commonly used command in real-world DevOps environments.

Delete Resources

- kubectl delete pod pod-name
 - kubectl delete -f deployment.yaml
-

6 Container Runtime & CRI Tools

View Logs

→ `kubectl logs pod-name`

If the Pod has multiple containers:

→ `kubectl logs pod-name -c container-name`

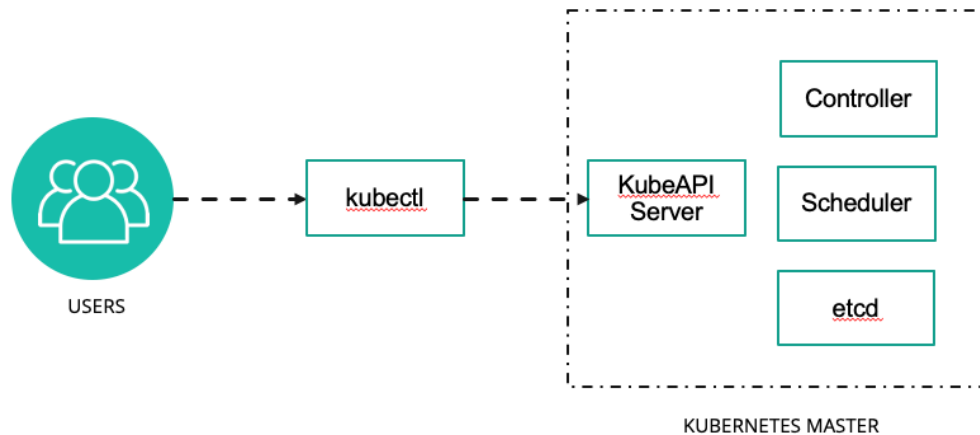
Execute Inside a Pod

→ `kubectl exec -it pod-name -- bash`

This allows you to access the container like SSH.

Scale a Deployment

→ `kubectl scale deployment app-name --replicas=3`



6 Container Runtime & CRI Tools

6.1 What is containerd?

6 Container Runtime & CRI Tools

containerd is a Container Runtime.

What does that mean?

It is the software responsible for actually running Containers on the Worker Node.

Previously, Kubernetes used Docker directly.

Now, Kubernetes communicates with the Container Runtime through something called CRI.

Examples of Container Runtimes:

- containerd
- CRI-O

Its Responsibilities:

- Pull images
- Create containers
- Start containers
- Stop containers
- Manage the container lifecycle

Important Note

containerd is not a CLI tool for regular users.

It runs underneath Kubernetes as the Container Runtime.

6.2 What is CRI?

CRI stands for Container Runtime Interface.

It is the interface between Kubernetes and the Container Runtime.

Kubernetes does not talk directly to containerd.

It communicates with it through CRI.

6.3 What is crictl?

crictl is a CLI tool used to interact directly with the Container Runtime.

7 Pods

If kubelet communicates with containerd through CRI, crictl can also communicate with it using the same interface.

It is mainly used for:

- Troubleshooting
- Debugging containers
- Viewing containers at the runtime level

Examples:

→ crictl ps

→ crictl images

✚ It shows containers from the Runtime perspective, not from the Kubernetes API perspective.

7 Pods

7.1 What is a Pod?

A Pod is the smallest deployable unit in Kubernetes.

Kubernetes does not run Containers directly.

It runs Pods, and each Pod contains one or more Containers.

7.2 What does a Pod contain?

A Pod includes:

- One or more Containers
- A shared network (same IP address)
- Shared storage (if volumes are defined)
- Metadata (name, labels, namespace)

All Containers inside the same Pod can communicate using:

7 Pods

→localhost

7.3 Pod Characteristics

- Each Pod has its own IP address
 - Containers inside a Pod share the same IP
 - Pods are ephemeral (temporary)
 - If a Pod crashes, its IP may change
-

7.4 Pod Types

1 Single-Container Pod

The most common type.

It contains one application container inside the Pod.

2 Multi-Container Pod

Contains more than one Container running together inside the same Pod.

These containers:

- Share the same network
 - Can communicate via localhost
 - Often follow the Sidecar pattern
-

7.5 Pod Lifecycle

A Pod goes through several phases:

- Pending
 - Running
 - Succeeded
 - Failed
-

8 Deployments

- CrashLoopBackOff
-

7.6 How to View Pods

To list all Pods:

→ `kubectl get pods`

To get detailed information about a specific Pod:

→ `kubectl describe pod pod-name`

To view logs of a Pod:

→ `kubectl logs pod-name`

7.7 Important Note

In real-world production environments,
we do not usually create Pods directly.

Instead, we create a Deployment that manages Pods.

8 Deployments

8.1 What is a Deployment?

A Deployment is a Kubernetes object used to manage Pods.

It ensures that:

- A specified number of Pods are running
 - Pods are automatically replaced if they fail
 - Updates are applied gradually (Rolling Updates)
 - Previous versions can be restored (Rollback)
-

8.2 What is a Pod Template?

Every Deployment contains a **Pod Template**.

The Pod Template defines:

- Container image
- Container name
- Ports
- Environment variables
- Volumes

The Deployment uses this template to create Pods.

8.3 How Deployment Works

You create a Deployment.

The Deployment creates a ReplicaSet.

The ReplicaSet creates and manages Pods.

Flow:

Deployment → ReplicaSet → Pods → Containers

8.4 Why Not Create a Pod Directly?

You can create a Pod directly like this:

→ `kubectl run nginx --image=nginx`

However, the problem is:

If the Pod crashes:

- **✗** No one will automatically replace it
- **✗** No automatic updates
- **✗** No real management

That is why in real-world environments:

8 Deployments

- ✓ We create a Deployment
 - ✓ Not a standalone Pod
-

8.5 Simple Explanation

If you create a Pod directly

= It is like a worker working alone

If you create a Deployment

= It is like a manager supervising workers

If a worker fails

The manager brings a replacement

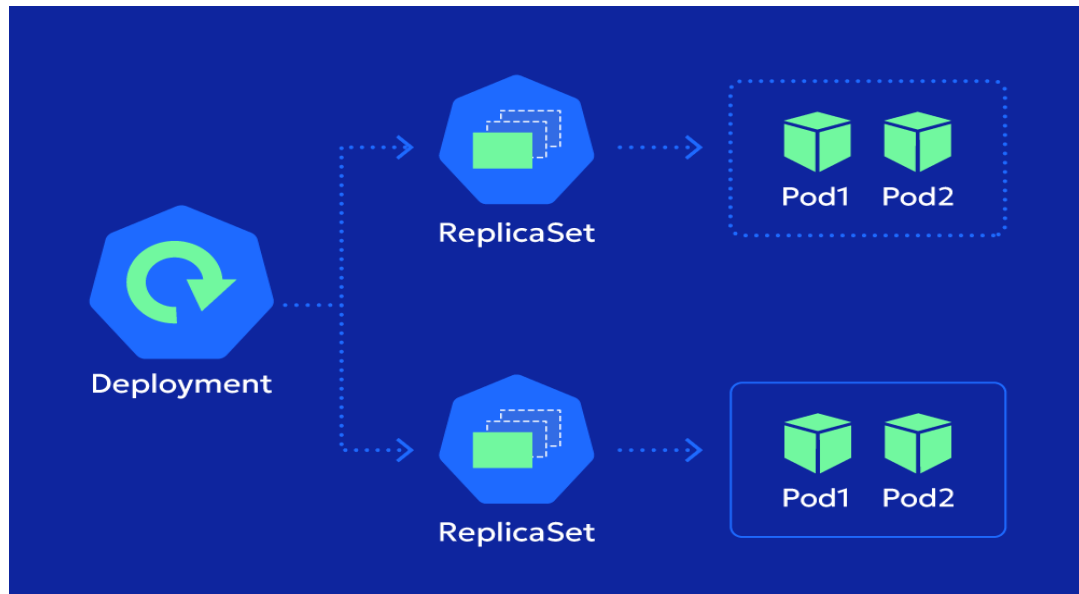
8.6 Summary

Pod:

- Smallest unit in Kubernetes
- Runs one or more Containers
- Temporary

Deployment:

- Manages Pods
- Maintains desired state
- Supports scaling and updates



9 YAML Basics

9.1 What is YAML?

YAML stands for **YAML Ain't Markup Language**.

It is a human-readable data format used to write configuration files.

YAML is widely used in:

- Kubernetes
- Docker Compose
- CI/CD pipelines
- Ansible
- GitHub Actions

9.2 YAML Structure Rules

YAML is based on **indentation (spaces)**, not brackets like JSON.

Important rules:

9 YAML Basics

- Indentation must use spaces (not tabs)
- Usually 2 spaces per level
- Structure depends on indentation

9.3 Dictionary (Key-Value)

A dictionary is the simplest structure in YAML.

Each key has a value.

```
name: Mohamed  
age: 22  
country: Egypt  
student: true
```

Format:

key: value

9.4 List (Array)

A list is a collection of items.

```
skills:  
- Kubernetes  
- Docker  
- Linux
```

Short format:

```
skills: [Kubernetes, Docker, Linux]
```

The - symbol indicates a new item in the list.

9.5 Dictionary of List

A key can contain a list as its value.

```
person:  
  name: Mohamed
```

9 YAML Basics

skills:

- Kubernetes
- Docker
- Linux

Here:

- person is a dictionary
 - skills is a list inside that dictionary
-

9.6 Dictionary of Dictionary

A key can contain another dictionary.

person:

name: Mohamed

address:

city: Tanta

country: Egypt

Here:

- person is a dictionary
- address is another dictionary inside it

Another example:

users:

user1:

name: Ali

age: 20

user2:

name: Sara

age: 21

9.7 Comments in YAML

Comments start with #.

name: Mohamed # This is a comment

9.8 Multi-line Strings

Using `|` preserves line breaks:

```
description: |
  This is line one
  This is line two
```

Using `>` converts lines into a single line:

```
description: >
  This is line one
  This is line two
```

9.9 Common YAML Mistakes

- Using tabs instead of spaces
- Wrong indentation
- Forgetting `-` in lists
- Mixing indentation levels



10 Kubernetes YAML Basics

10.1 What is Kubernetes YAML?

Kubernetes YAML is a configuration file used to define Kubernetes objects (resources).

You write the object details in a YAML file, then apply it using:

→ `kubectl apply -f file.yaml`

You can also create resources using:

→ `kubectl create -f file.yaml`

10.2 The Common Structure of a Kubernetes YAML File

Most Kubernetes YAML files contain these main fields:

- `apiVersion`
- `kind`
- `metadata`
- `spec`

```
apiVersion: ...  
kind: ...  
metadata:  
  name: ...  
spec:  
  ...
```

10.3 `apiVersion`

Defines which Kubernetes API version is used for the object.

Examples:

10 Kubernetes YAML Basics

- v1 (Pod, Service)
 - apps/v1 (Deployment, Replica Set)
-

10.4 kind

Defines the type of Kubernetes object.

Examples:

- Pod
 - Deployment
 - Service
 - ConfigMap
 - Secret
-

10.5 metadata

Contains identifying information about the object.

Common fields:

- name
- labels
- namespace

Example:

```
metadata:
  name: nginx-deployment
  labels:
    app: nginx
```

10.6 spec

The most important section.

It contains the actual configuration of the object.

Examples:

- containers
- image
- ports
- replicas
- selector
- environment variables
- volumes

10.7 Example: Pod YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
```

10.8 Example: Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

10.9 Important Notes

- YAML indentation is critical (use spaces, not tabs).
- In Deployments, the Pod is defined inside template.
- selector.matchLabels must match template.metadata.labels.

10.10 Useful Commands with YAML

Create resources from a YAML file:

1 Replication in Kubernetes

→ `kubectl apply -f file.yaml`

Apply changes to resources:

→ `kubectl apply -f file.yaml`

Delete resources created by the YAML file:

→ `kubectl delete -f file.yaml`

1 Replication in Kubernetes

1.1 What is Replication?

Replication means running **multiple identical Pods of the same application**.

Instead of running a single Pod, Kubernetes can run several Pods that provide the same service.

Example:

1 Pod → single instance of the application

3 Pods → three identical instances

Benefits of replication:

- High Availability
- Fault Tolerance
- Load Distribution
- Better performance under heavy traffic

If one Pod fails, other Pods can still serve the application.

1.2 ReplicationController

A ReplicationController is a Kubernetes object that ensures a specified number of Pods are always running.

1 Replication in Kubernetes

If a Pod crashes or is deleted, the ReplicationController automatically creates a new Pod to replace it.

Main responsibilities:

- Maintain the desired number of Pods
- Replace failed Pods
- Remove extra Pods if the number exceeds the desired count

Example YAML:

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Explanation:

- replicas → number of Pods that should be running
- selector → identifies which Pods belong to the controller
- template → describes the Pod that will be created

1.3 ReplicaSet

ReplicaSet is the **modern replacement for ReplicationController**.

1 **1** Replication in Kubernetes

It performs the same basic function:

Maintaining the desired number of Pods.

However, ReplicaSet supports **more advanced label selectors**.

In modern Kubernetes, ReplicaSets are usually created and managed by **Deployments**.

Example YAML:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

Important concept:

Deployment → manages ReplicaSet → manages Pods

2 **2** Scaling in Kubernetes

2.1 What is Scaling?

Scaling means **changing the number of running Pods** for an application.

1 Replication in Kubernetes

Scaling can be used when:

- Application traffic increases
- More resources are required
- Load balancing is needed

Example:

Low traffic → 1 Pod

High traffic → 5 Pods

2.2 Manual Scaling

Manual scaling means the user manually changes the number of Pods.

Example command:



Bash 

```
kubectl scale deployment nginx-deployment --replicas=5
```

This command instructs Kubernetes to run **5 Pods** of the application.

Kubernetes will:

- Create new Pods if the number is too low
- Remove Pods if the number is too high

2.3 Scaling Using YAML

Scaling can also be defined in the Deployment YAML file.

Example:



YAML 

```
spec:  
  replicas: 3
```

This means Kubernetes should always maintain **3 running Pods**.

If a Pod crashes:

1 **1** Replication in Kubernetes

Kubernetes automatically creates a new Pod to maintain the desired number.

2.4 Auto Scaling (Overview)

Kubernetes can also scale Pods automatically using **Horizontal Pod Autoscaler (HPA)**.

HPA monitors metrics such as:

- CPU usage
- Memory usage

If the usage increases, Kubernetes creates more Pods.

If the usage decreases, Kubernetes removes extra Pods.

Example command:

```
 Bash   
kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=1 --max=10
```

Meaning:

- Minimum Pods = 1
 - Maximum Pods = 10
 - Target CPU usage = 50%
-

3 Labels

3.1 What are Labels?

Labels are **key-value pairs attached to Kubernetes objects**.

They are used to identify, organize, and group resources.

Labels can be applied to:

- Pods
- Nodes

1 Replication in Kubernetes

- Deployments
- Services

Example:

```
 YAML </>  
labels:  
  app: nginx  
  environment: production
```

Here:

- app is the key
 - nginx is the value
-

3.2 Why Labels are Important?

Labels allow Kubernetes to:

- Organize resources
- Group related objects
- Allow controllers to find the Pods they manage
- Enable Services to connect to the correct Pods

Without labels, Kubernetes would not know which Pods belong to which controller or service.

3.3 Example Pod with Labels

1 Replication in Kubernetes

 YAML 

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx
```

:This Pod contains two labels

 Code 

```
app: nginx
tier: frontend
```

Selectors

4.1 What is a Selector?

A Selector is used by Kubernetes objects to **find and select Pods based on Labels**.

Controllers such as:

- Deployments
- ReplicaSets
- Services

use selectors to identify the Pods they manage.

4.2 matchLabels

matchLabels is the simplest type of label selector.

It selects Pods that have specific labels.

Example:

1 **1** Replication in Kubernetes

```
selector:
  matchLabels:
    app: nginx
```

:This selector will match all Pods that contain

```
app: nginx
```

4.3 Example Deployment Using Labels and Selectors

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

Explanation:

1. The Deployment uses a selector to find Pods with the label `app: nginx`.
2. The template creates Pods that contain the label `app: nginx`.
3. Because the labels match the selector, the Deployment manages those Pods.

4.4 Important Rule

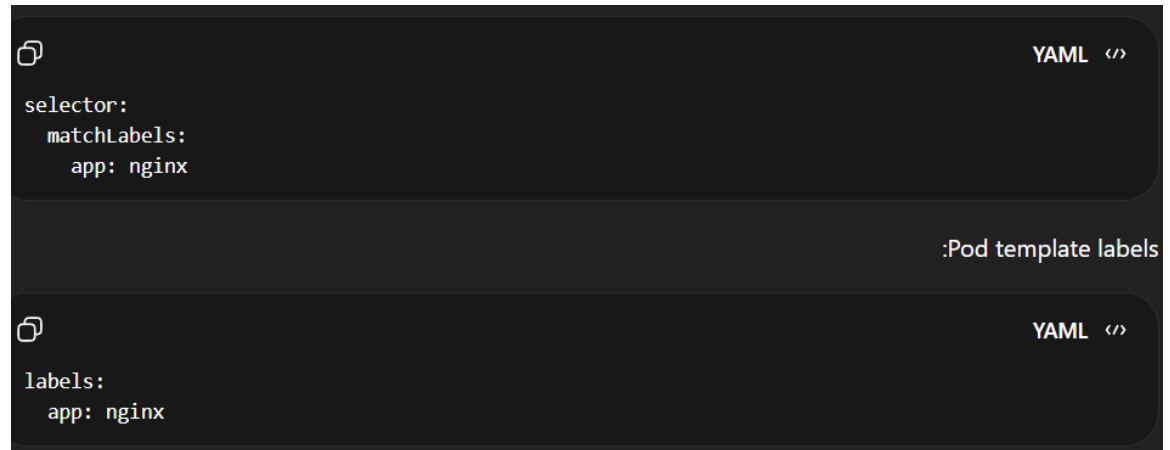
The selector must match the Pod labels.

1 **1** Replication in Kubernetes

Example:

Selector:

selector:



The image shows two code snippets in a dark-themed editor. The top snippet is a YAML configuration for a selector, with a 'YAML' label and a copy icon in the top right corner. The bottom snippet is a YAML configuration for labels, also with a 'YAML' label and a copy icon in the top right corner. A label ':Pod template labels' is positioned between the two snippets on the right side.

```
selector:
  matchLabels:
    app: nginx
```

:Pod template labels

```
labels:
  app: nginx
```

If these labels do not match, the Deployment cannot manage the Pods correctly.

5 Summary

Replication:

- Ensures multiple copies of a Pod are running.

ReplicaSet:

- Maintains the desired number of Pods.

Scaling:

- Increases or decreases the number of Pods.

Labels:

- Key–value pairs used to identify and organize Kubernetes objects.

Selectors:

- Used to find Pods based on their Labels.

matchLabels:

- The simplest type of label selector.

1 **2** Deployments in Kubernetes

12.1 What is a Deployment?

A Deployment is a Kubernetes object used to **manage and control Pods**.

Instead of creating Pods directly, we create a Deployment that manages Pods automatically.

The Deployment ensures that the application runs with the desired configuration and number of Pods.

12.2 Why Use Deployments?

Deployments provide several important features:

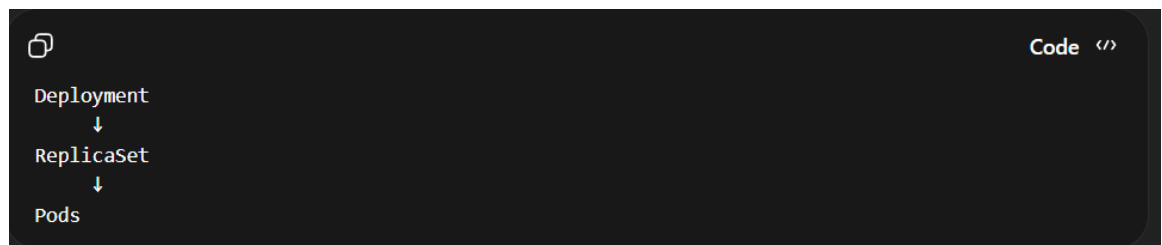
- Automatic Pod management
- Scaling applications
- Rolling updates
- Rollback to previous versions
- High availability

Because of these features, Deployments are the **most commonly used object in Kubernetes** for running applications.

12.3 How Deployment Works

A Deployment manages Pods through ReplicaSets.

The architecture works as follows:



Explanation:

1 **2** Deployments in Kubernetes

- The Deployment defines the desired state.
- The ReplicaSet ensures the correct number of Pods are running.
- The Pods run the application containers.

12.4 Deployment YAML Example

Example of a simple Deployment configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Explanation:

- replicas defines the number of Pods.
- selector identifies which Pods belong to this Deployment.
- template defines the Pod configuration.

12.5 Creating a Deployment

To create a Deployment from a YAML file:

1 **2** Deployments in Kubernetes

```
kubectl create -f deployment.yaml
```

or

```
kubectl apply -f deployment.yaml
```

12.6 Viewing Deployments

```
kubectl get deployments
```

:To list all Deployments

```
kubectl describe deployment nginx-deployment
```

:To view detailed information

12.7 Updating a Deployment

You can update the application by changing the container image.

Example:

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.25
```

Kubernetes performs a **rolling update**, meaning Pods are updated gradually without downtime.

12.8 Rolling Updates

Rolling updates allow updating the application without stopping the service.

The process works as follows:

1. Kubernetes creates new Pods with the new version.

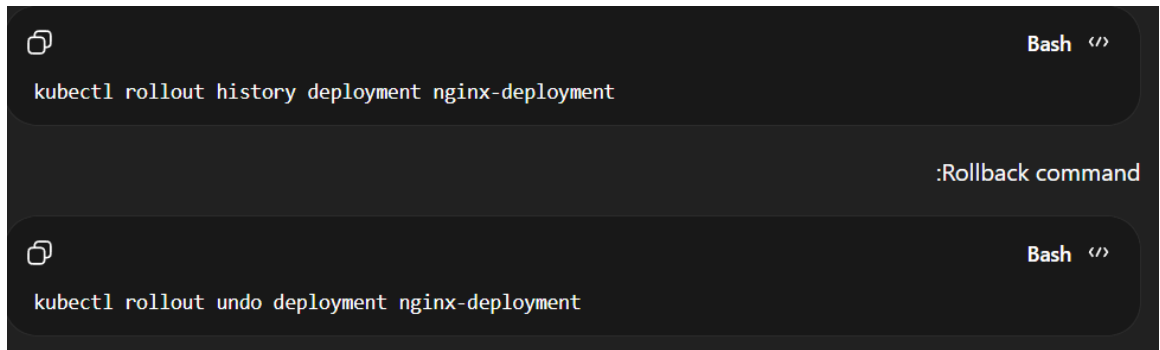
1 **2** Deployments in Kubernetes

2. Old Pods are terminated gradually.
3. The service continues running during the update.

12.9 Rollback

If a new version causes problems, Kubernetes allows rolling back to a previous version.

Check deployment history:



```
kubectl rollout history deployment nginx-deployment
```

:Rollback command

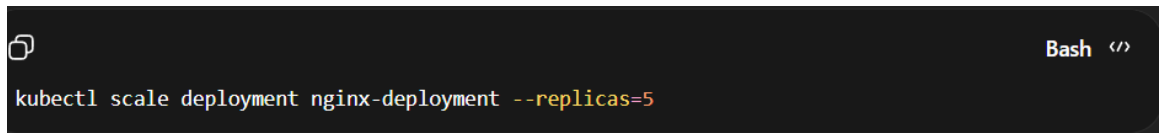
```
kubectl rollout undo deployment nginx-deployment
```

The image shows a terminal window with a dark background. It contains two code blocks. The first block shows the command `kubectl rollout history deployment nginx-deployment`. The second block shows the command `kubectl rollout undo deployment nginx-deployment`. Between the two blocks, the text `:Rollback command` is displayed. Each code block has a copy icon on the left and 'Bash </>' on the right.

12.10 Scaling a Deployment

You can change the number of Pods using scaling.

Example:



```
kubectl scale deployment nginx-deployment --replicas=5
```

The image shows a terminal window with a dark background. It contains a single code block with the command `kubectl scale deployment nginx-deployment --replicas=5`. The code block has a copy icon on the left and 'Bash </>' on the right.

This command increases the number of Pods to 5.

12.11 Summary

Deployment:

- Manages Pods automatically
- Uses ReplicaSets to maintain the desired number of Pods
- Supports scaling and updates

Deployment architecture:

1 **3** Deployment Strategies and Updates



Code </>

Deployment → ReplicaSet → Pods → Containers

Deployments are the **recommended way to run applications in Kubernetes**.

1 **3** Deployment Strategies and Updates

13.1 Updating Applications in Kubernetes

When an application needs a new version (for example updating the container image), Kubernetes provides mechanisms to update running Pods safely.

Deployment strategies define **how Pods are replaced during updates**.

The two main strategies are:

- Rolling Update
- Recreate

Kubernetes also provides a mechanism called:

- Rollback

Rollback allows reverting to a previous working version of the application.

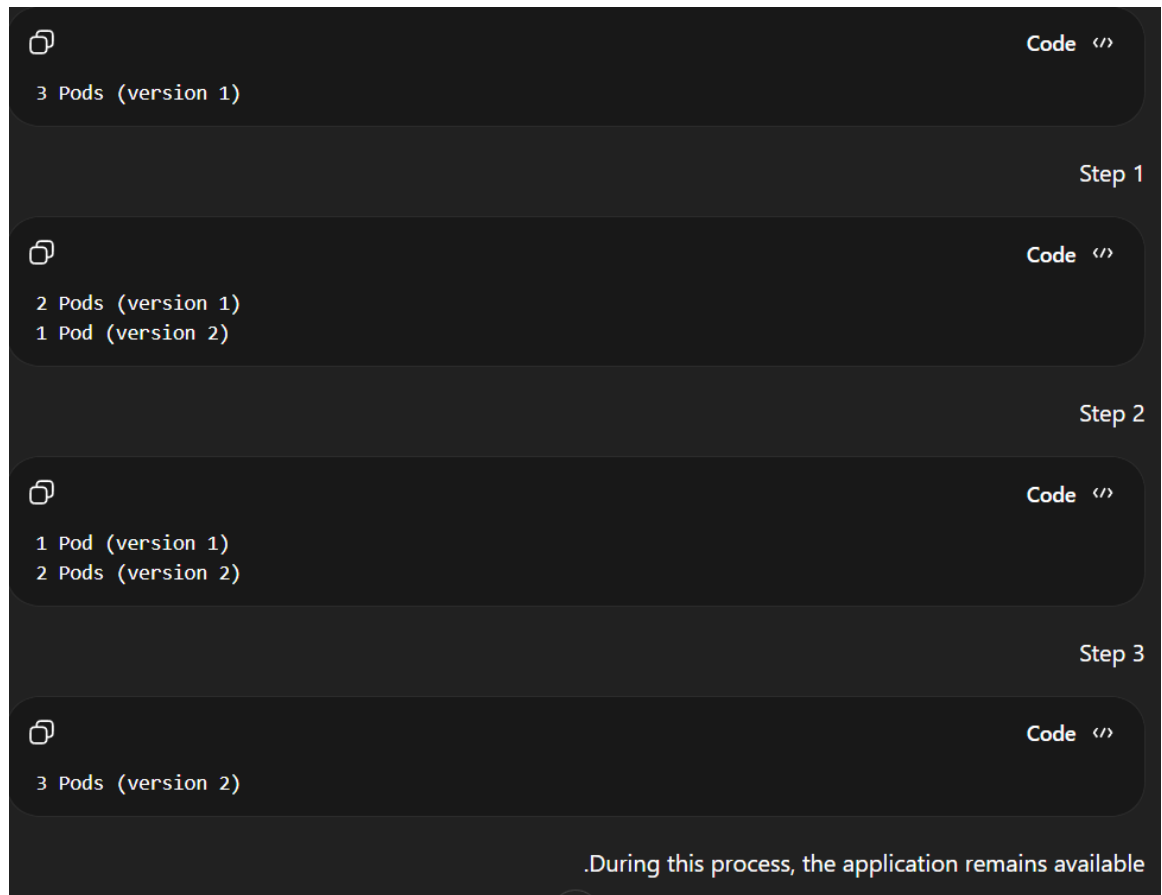
13.2 Rolling Update

Rolling Update is the **default update strategy in Kubernetes Deployments**.

Instead of stopping all Pods at once, Kubernetes replaces Pods **gradually**.

Example:

1 **3** Deployment Strategies and Updates



13.3 Advantages of Rolling Update

Rolling updates provide several benefits:

- No downtime
- Continuous service availability
- Gradual replacement of Pods
- Safer application updates

Because of these advantages, Rolling Update is the **default strategy in Kubernetes**.

13.4 Rolling Update Configuration

Rolling updates can be configured inside the Deployment YAML file.

Example:

1 **3** Deployment Strategies and Updates

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

Explanation:

- maxUnavailable → maximum number of Pods that can be unavailable during the update
- maxSurge → maximum number of extra Pods that can be created during the update

These parameters control **how fast the update occurs**.

13.5 maxUnavailable

maxUnavailable defines the **maximum number of Pods that can be unavailable during the update**.

Example:

Deployment replicas:

```
replicas = 4
```

:Configuration

```
maxUnavailable: 1
```

.This means Kubernetes allows only 1 Pod to stop during the update

:Therefore, at least

```
3 Pods must remain running
```

.This ensures the application continues serving requests

13.6 maxSurge

maxSurge defines the **maximum number of extra Pods that Kubernetes can create during the update.**

Example:

Deployment replicas:

replicas = 4

Configuration:

maxSurge: 1

Kubernetes can temporarily create:

5 Pods

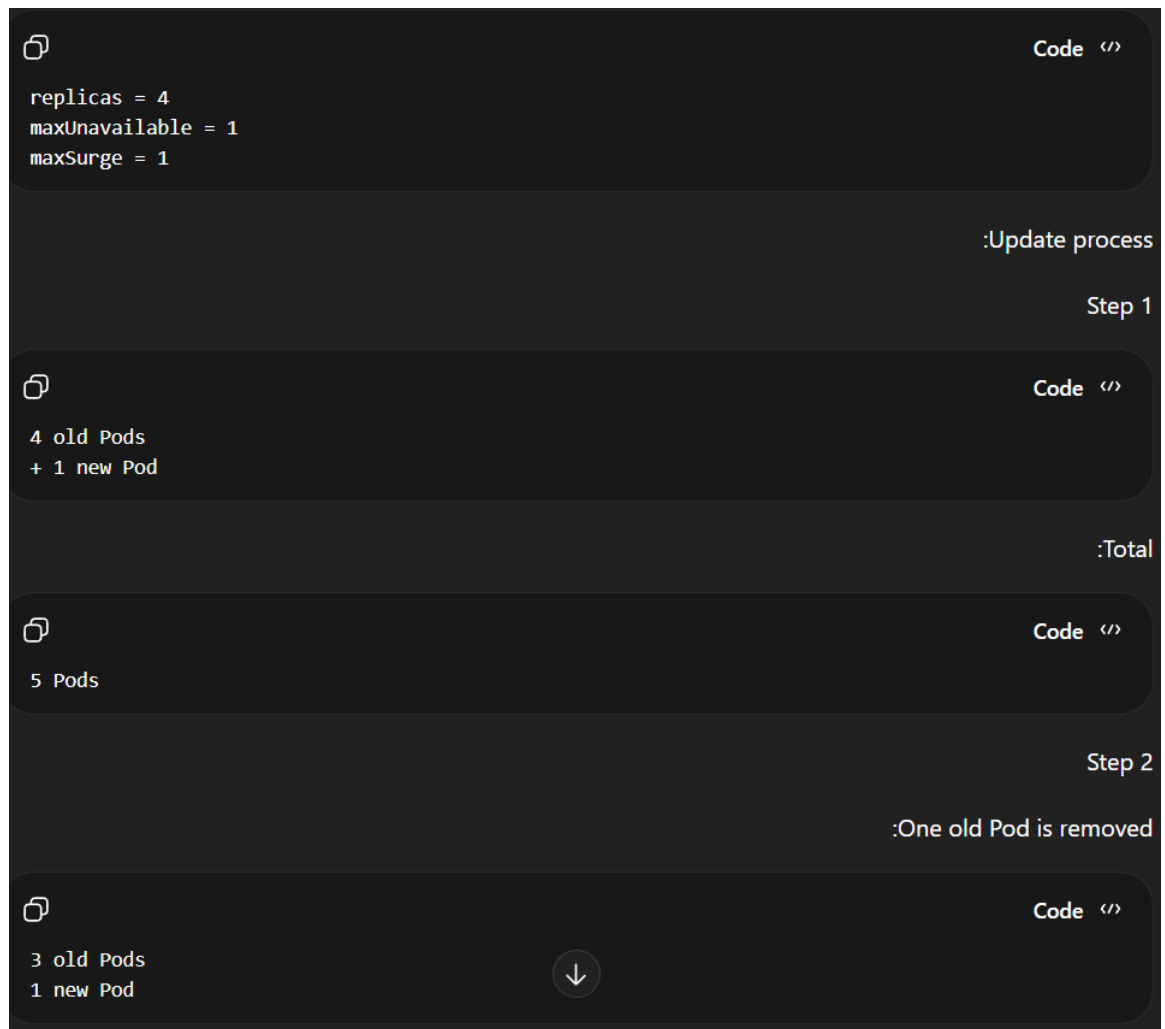
(4 existing Pods + 1 new Pod)

After the new Pod becomes ready, Kubernetes deletes an old Pod.

13.7 Example Rolling Update Process

Deployment configuration:

1 **3** Deployment Strategies and Updates



The process continues until all Pods run the new version.

13.8 Rolling Update YAML Example

1 **3** Deployment Strategies and Updates

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

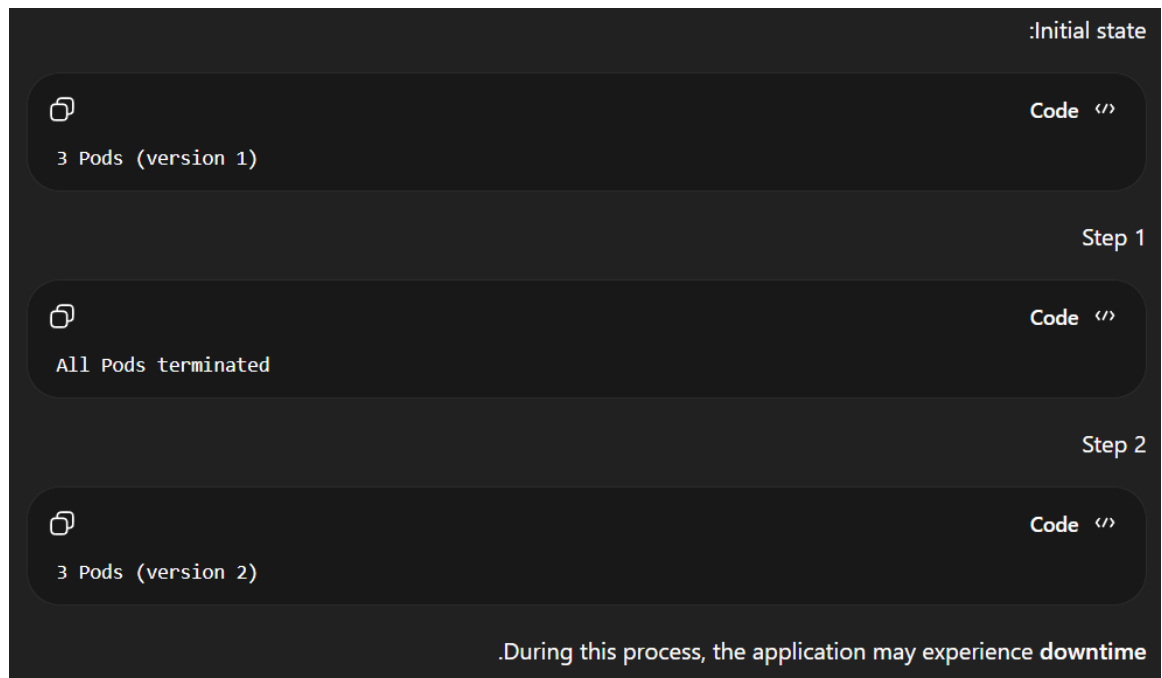
13.9 Recreate Strategy

The Recreate strategy works differently from Rolling Update.

Instead of replacing Pods gradually, Kubernetes **terminates all existing Pods first**, then creates new Pods.

Example:

1 **3** Deployment Strategies and Updates



13.10 Recreate Configuration

Example configuration:

```
strategy:
  type: Recreate
```

This tells Kubernetes to delete all existing Pods before creating the new version.

13.11 Rolling Update vs Recreate

Feature	Rolling Update	Recreate
Pod replacement	Gradual	All at once
Downtime	No	Possible
Service availability	High	Low during update
Default strategy	Yes	No

Rolling Update is recommended for **production environments**.

13.12 Rollback

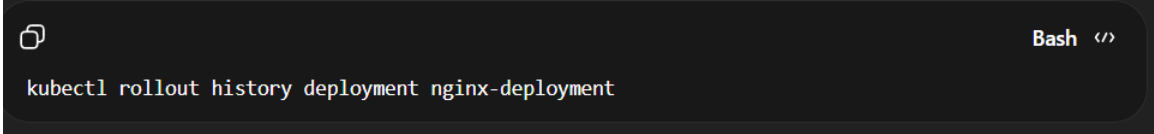
Rollback allows Kubernetes to **restore a previous version of a Deployment** if a new update fails.

This is useful when:

- A new container image has bugs
- The application fails after the update
- Performance problems appear

13.13 Checking Deployment History

To see the update history of a Deployment:



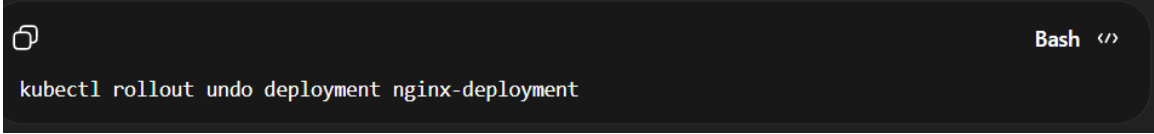
```
kubectl rollout history deployment nginx-deployment
```

A terminal window with a dark background. In the top right corner, it says "Bash" followed by a code icon. The command `kubectl rollout history deployment nginx-deployment` is entered in the terminal.

This command shows all previous versions of the Deployment.

13.14 Rolling Back to the Previous Version

To revert to the previous version:



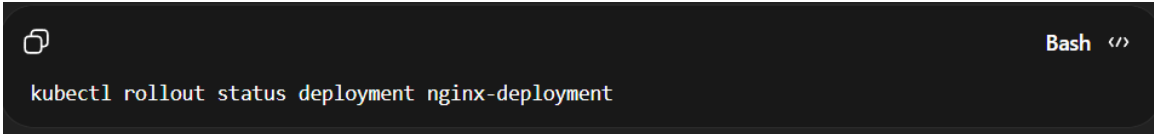
```
kubectl rollout undo deployment nginx-deployment
```

A terminal window with a dark background. In the top right corner, it says "Bash" followed by a code icon. The command `kubectl rollout undo deployment nginx-deployment` is entered in the terminal.

Kubernetes will restore the previous working version.

13.15 Monitoring Rollout Status

You can monitor the progress of a deployment update using:



```
kubectl rollout status deployment nginx-deployment
```

A terminal window with a dark background. In the top right corner, it says "Bash" followed by a code icon. The command `kubectl rollout status deployment nginx-deployment` is entered in the terminal.

This command shows whether the update is still in progress or completed.

13.16 Default Values in Kubernetes

If maxSurge and maxUnavailable are not specified, Kubernetes uses default values:



Code 

```
maxUnavailable = 25%  
maxSurge = 25%
```

These percentages are calculated based on the number of replicas.

1 Rollout and Rollback in Kubernetes

14.1 What is a Rollout?

A **rollout** refers to the process of **deploying or updating an application** in Kubernetes.

When you update a Deployment (for example by changing the container image), Kubernetes performs a rollout.

During a rollout, Kubernetes gradually replaces old Pods with new Pods according to the Deployment strategy.

Example update:

nginx:1.24 → nginx:1.25

Kubernetes performs a **Rolling Update** where Pods are updated gradually.

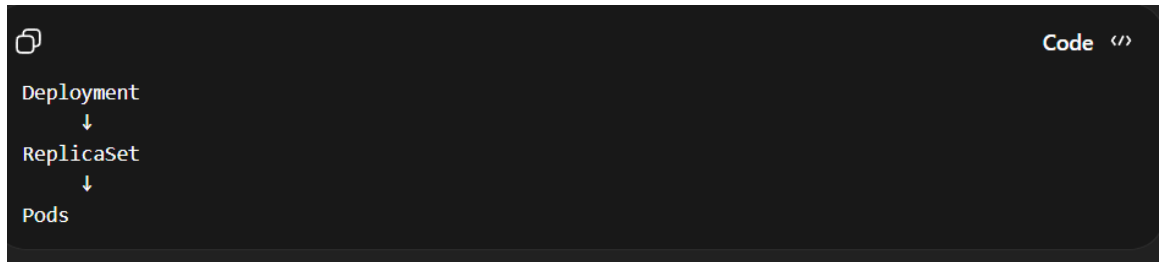
14.2 Rollout Process

When a Deployment rollout happens:

1. Kubernetes creates new Pods with the updated configuration.
2. Old Pods are terminated gradually.
3. The ReplicaSet ensures the desired number of Pods is maintained.

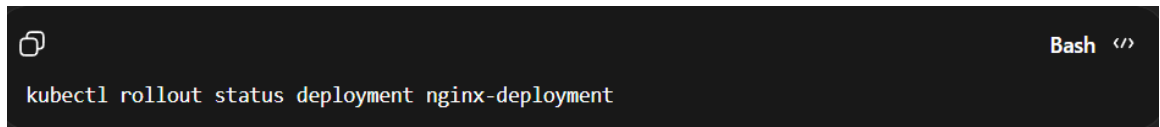
1 Rollout and Rollback in Kubernetes

Architecture during rollout:



14.3 Checking Rollout Status

You can monitor the progress of a rollout using:



```
kubectl rollout status deployment nginx-deployment
```

A terminal window showing the command `kubectl rollout status deployment nginx-deployment` used to monitor the progress of a rollout.

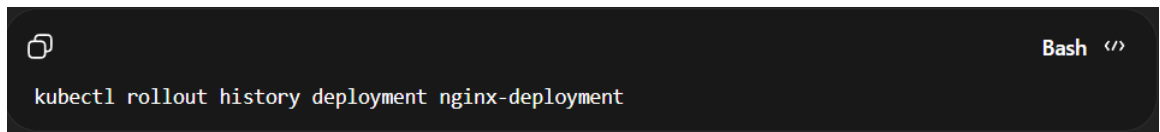
This command shows whether the update is:

- In progress
- Completed
- Failed

14.4 Viewing Rollout History

Kubernetes stores a history of Deployment revisions.

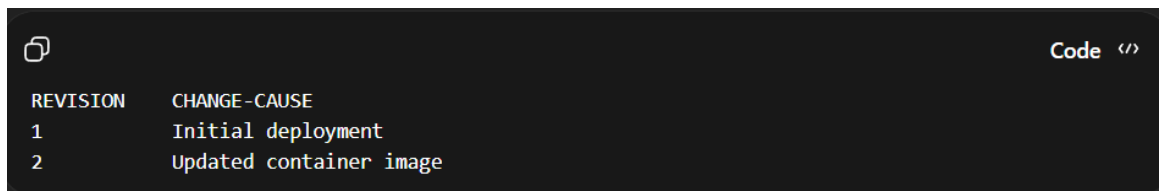
You can view previous updates using:



```
kubectl rollout history deployment nginx-deployment
```

A terminal window showing the command `kubectl rollout history deployment nginx-deployment` used to view the history of deployment revisions.

Example output:



```
REVISION    CHANGE-CAUSE
1           Initial deployment
2           Updated container image
```

The example output of the `kubectl rollout history` command, showing two revisions. Revision 1 is the 'Initial deployment' and Revision 2 is the 'Updated container image'.

Each revision represents a version of the Deployment.

14.5 What is a Rollback?

A **rollback** is the process of **reverting a Deployment to a previous version**.

Rollback is useful when a new version of the application causes problems such as:

- Bugs in the application
- Application crashes
- Performance issues

Instead of manually fixing the deployment, Kubernetes allows restoring the previous working version.

14.6 Performing a Rollback

To revert a Deployment to the previous version:

```
 Bash   
kubect1 rollout undo deployment nginx-deployment
```

Kubernetes restores the previous ReplicaSet and replaces the current Pods.

14.7 Rolling Back to a Specific Revision

You can also rollback to a specific revision.

Example:

```
 Bash   
kubect1 rollout undo deployment nginx-deployment --to-revision=1
```

This command restores revision 1 of the Deployment.

14.8 The --record Option

In older versions of Kubernetes, the --record flag was used to record the command that triggered a Deployment update.

Example:

1 5 Kubernetes Networking



Bash ↗

```
kubectl set image deployment nginx-deployment nginx=nginx:1.25 --record
```

This would store the command in the rollout history.

Then when running:



Bash ↗

```
kubectl rollout history deployment nginx-deployment
```

You could see what command caused each revision.

Example:



Code ↗

REVISION	CHANGE-CAUSE
1	kubectl apply -f deployment.yaml
2	kubectl set image deployment nginx-deployment nginx=nginx:1.25

14.9 Important Note

The `--record` flag has been **deprecated and removed in newer Kubernetes versions**.

Modern Kubernetes automatically tracks Deployment revisions without requiring this flag.

1 5 Kubernetes Networking

15.1 Why Networking is Needed in Kubernetes

In Kubernetes, applications run inside **Pods**.

Modern applications are usually composed of multiple components such as:

- Frontend
- Backend
- Database

1 **5** Kubernetes Networking

- APIs

Each component may run in a different Pod.

Therefore Kubernetes needs a **networking model that allows Pods to communicate with each other**.

15.2 The Kubernetes Networking Model

Kubernetes networking follows three fundamental rules.

Rule 1: Every Pod gets its own IP

Each Pod receives a **unique IP address** inside the cluster.

Example:

Pod A → 10.244.0.5

Pod B → 10.244.0.6

Pod C → 10.244.1.2

The Pod behaves like a **separate machine on the network**.

Containers inside the Pod do not get separate IP addresses.

Rule 2: Pods can communicate without NAT

Any Pod can communicate with any other Pod directly using its IP address.

Example:

Pod A (10.244.0.5) → Pod B (10.244.0.6)

No NAT or port forwarding is required.

Rule 3: Containers inside the same Pod share the network

Containers inside the same Pod share:

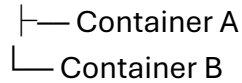
- The same IP
- The same network namespace

1 **5** Kubernetes Networking

- The same localhost

Example Pod:

Pod (IP: 10.244.0.5)



Communication:

Container A → localhost:3000 → Container B

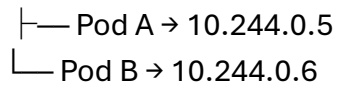
Because they share the same network environment.

15.3 Pod Network

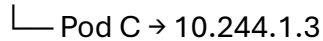
The **Pod Network** connects all Pods across the cluster.

Diagram:

Node 1



Node 2



Pods communicate using their Pod IPs:

Pod A → 10.244.0.5



Pod C → 10.244.1.3

15.4 Node Network

Each Kubernetes node has its own IP address.

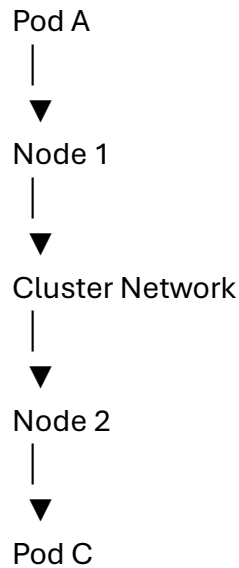
Example:

Node 1 → 192.168.1.10

Node 2 → 192.168.1.11

Pods running on different nodes communicate through the node network.

Traffic example:



15.5 CNI (Container Network Interface)

CNI plugins are responsible for implementing Kubernetes networking.

Responsibilities:

- Assign Pod IP addresses
- Connect Pods across nodes
- Configure routing between networks

Common CNI plugins:

- Calico
 - Flannel
 - Weave
 - Cilium
-

1 Kubernetes Services

16.1 Why Services are Needed

Pods are **ephemeral**, meaning they can be:

- Created
- Deleted
- Replaced

Their IP addresses may change.

Example:

Pod A → 10.244.0.5

Pod A recreated → 10.244.0.9

This would break application communication.

Solution:

Service

A Service provides a **stable IP and DNS name**.

16.2 How Services Work

Services connect to Pods using:

Labels + Selectors

Example Pod label:

```
YAML </>
labels:
  app: nginx

:Service selector

YAML </>
selector:
  app: nginx
```

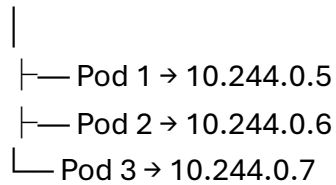
The Service forwards traffic to all Pods with that label.

16.3 Service Load Balancing

A Service automatically distributes traffic between Pods.

Example:

Service



Each request may go to a different Pod.

16.4 Types of Services

Kubernetes provides several Service types.

16.4.1 ClusterIP

ClusterIP is the **default Service type**.

It allows communication **only inside the cluster**.

Example architecture:

Frontend Pod



1 Kubernetes Services

Backend Service (ClusterIP)



Backend Pods

Example YAML:

```
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 8080
```

16.4.2 NodePort

NodePort exposes the application outside the cluster.

Kubernetes opens a port on every node.

Example:

User



NodeIP:30007



Service



Pods

Example YAML:

type: NodePort

Access example:

http://192.168.1.10:30007

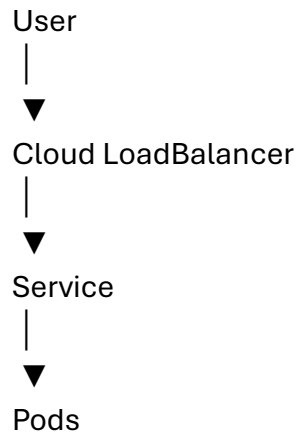
16.4.3 LoadBalancer

LoadBalancer is used in cloud environments.

Examples:

- AWS
- Azure
- GCP

Traffic flow:



Example:

Public IP → 35.200.10.5

16.4.4 ExternalName

ExternalName maps a Kubernetes Service to an external DNS name.

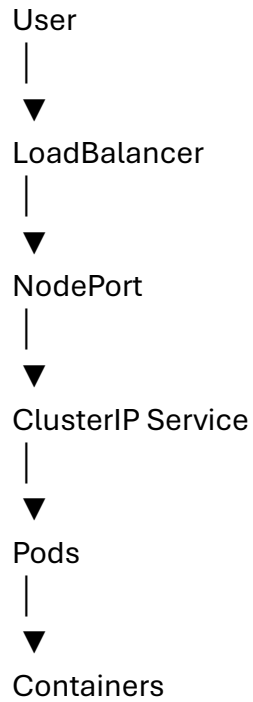
Example:

Service → external database

Full Kubernetes Networking Flow

The complete traffic path may look like this:

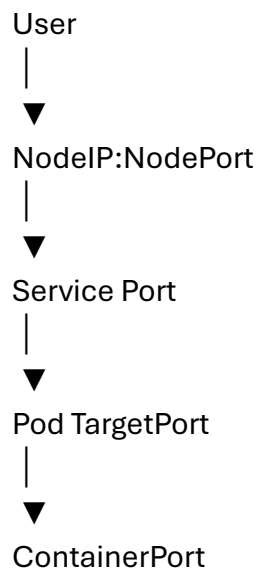
1 **8** Port Mapping in Kubernetes



1 **8** Port Mapping in Kubernetes

Traffic passes through several port layers.

Diagram:



18.1 ContainerPort

The port where the application runs inside the container.

Example:

containerPort: 8080

18.2 TargetPort

The port where the Service forwards traffic inside the Pod.

Example:

targetPort: 8080

18.3 Service Port

The port exposed by the Service inside the cluster.

Example:

port: 80

Pods inside the cluster access the service using:

service-name:80

18.4 NodePort

The external port opened on each node.

Example:

nodePort: 30007

External access:

NodeIP:30007

18.5 Example YAML

1 **8** Port Mapping in Kubernetes

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30007
```

18.6 Traffic Flow Example

User
|
▼
192.168.1.10:30007
|
▼
Service:80
|
▼
Pod:8080
|
▼
Container:8080

Simple Rule to Remember

nodePort → outside the cluster
port → inside the cluster
targetPort → inside the Pod
containerPort → inside the container

2 **1** Practical Networking Scenario

21.1 Application Architecture Example

Suppose we have a simple web application composed of:

- **Frontend** → Web interface
- **Backend API** → Business logic
- **Database** → Stores data

Each component runs inside its own Pods.

Architecture:

Frontend Pods

Backend Pods

Database Pod

21.2 Pod Layout in the Cluster

Assume the cluster has two nodes.

Node 1 → 192.168.1.10

Node 2 → 192.168.1.11

Pods running in the cluster:

Node 1

└─ Frontend Pod → 10.244.0.5

└─ Backend Pod → 10.244.0.6

Node 2

└─ Frontend Pod → 10.244.1.3

└─ Database Pod → 10.244.1.4

Each Pod has its own IP in the **Pod Network**.

21.3 Internal Communication Using ClusterIP

The frontend needs to communicate with the backend API.

Instead of using Pod IPs directly, Kubernetes creates a **Service**.

Service example:

1 8 Port Mapping in Kubernetes

Backend Service

ClusterIP → 10.96.0.20

Communication flow:

Frontend Pod



backend-service:80



Backend Pods

Example:

10.244.0.6

10.244.1.7

The Service performs **load balancing** between backend Pods.

21.4 Database Access Inside the Cluster

The backend API needs to access the database.

A database service is created.

Example:

Database Service

ClusterIP → 10.96.0.30

Communication flow:

Backend Pod



database-service:5432



Database Pod

21.5 External Access Using NodePort

1 **8** Port Mapping in Kubernetes

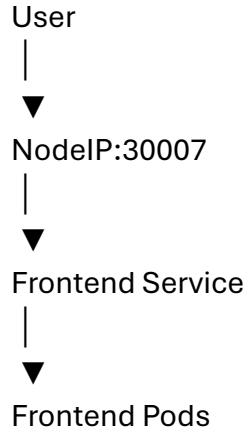
Users on the internet need to access the frontend application.

A **NodePort Service** is created.

Example:

NodePort → 30007

Traffic flow:



Example:

http://192.168.1.10:30007

Kubernetes routes the request to one of the frontend Pods.

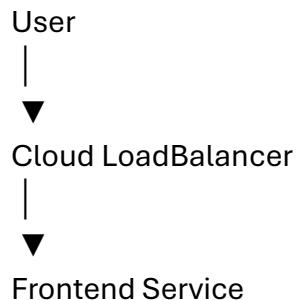
21.6 External Access Using LoadBalancer

In cloud environments, Kubernetes can create a **LoadBalancer Service**.

Example public IP:

35.200.10.5

Traffic flow:



1 **8** Port Mapping in Kubernetes

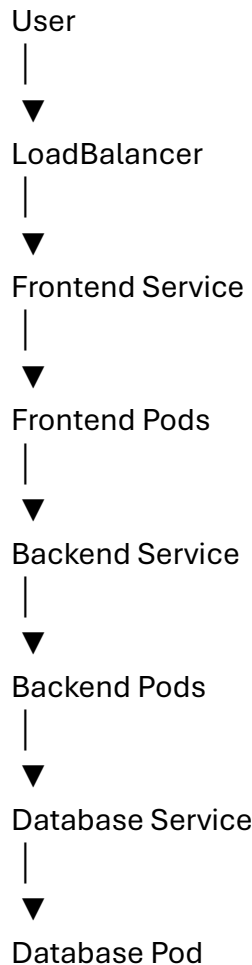


Frontend Pods

This allows the application to be accessed directly from the internet.

21.7 Complete Application Network Flow

The full networking path of the application looks like this:



21.8 Important Observations

From this scenario we can see that:

- Pods communicate using **Pod IPs internally**
- Services provide **stable endpoints**

2 **2** Microservices Architecture Example (Voting App)

- Services use **labels and selectors to find Pods**
 - Services perform **load balancing automatically**
-

2 **2** Microservices Architecture Example (Voting App)

22.1 What is a Microservice in this Example?

In the **Voting App**, the application is divided into multiple small services.

Each service has:

- Its own **container**
- Its own **Pods**
- Its own **Deployment**
- Its own **Service**

Each service performs **one specific task**.

22.2 Components of the Voting Application

The application consists of five main services:



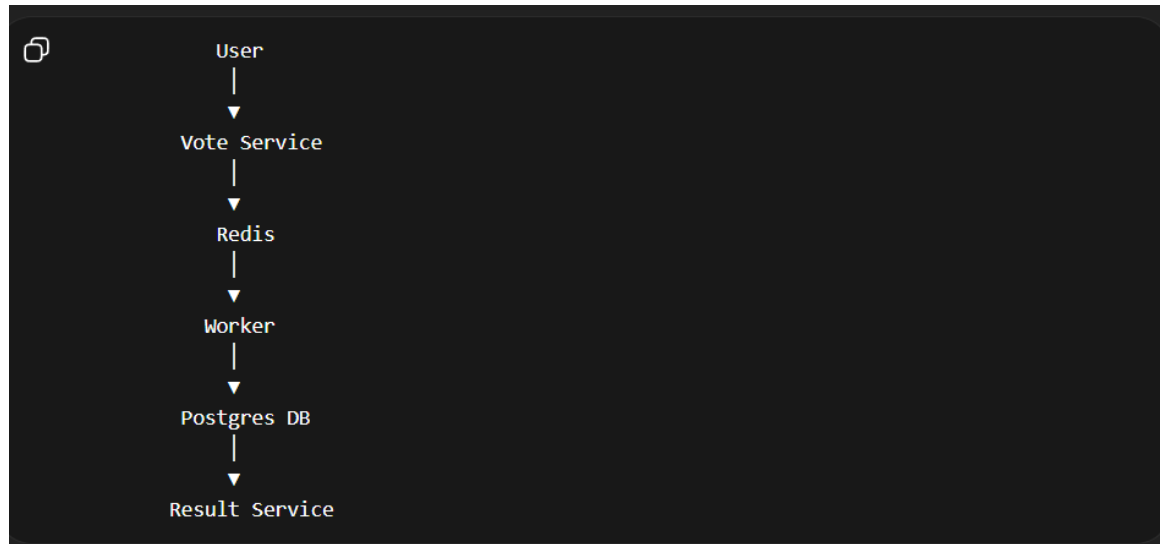
Plain text 

```
vote service  
redis  
worker service  
postgres database  
result service
```

Each component is a **separate microservice**.

22.3 Architecture Diagram

2 **2** Microservices Architecture Example (Voting App)



22.4 Role of Each Microservice

1 Vote Service

This service provides the **web interface where users vote**.

Example options:

Vote for Cats

Vote for Dogs

The vote service does **not store votes in the database directly**.

Instead it sends the vote to **Redis**.

2 Redis

Redis acts as a **temporary storage or message queue**.

Example stored messages:

vote: cats

vote: dogs

vote: cats

Redis is used because it is **very fast for handling large numbers of requests**.

2 **2** Microservices Architecture Example (Voting App)

3 Worker Service

The worker continuously reads votes from Redis.

It processes the data and stores the final result in **Postgres database**.

Flow:

Redis → Worker → Database

This separates **processing logic from user interaction**.

4 Postgres Database

Postgres stores the **permanent voting results**.

Example:

Cats: 150 votes

Dogs: 120 votes

5 Result Service

The result service displays the final voting results.

Example:

Cats: 150

Dogs: 120

It reads data from **Postgres database**.

22.5 Step-by-Step Request Flow

When a user votes, the system processes the request as follows:

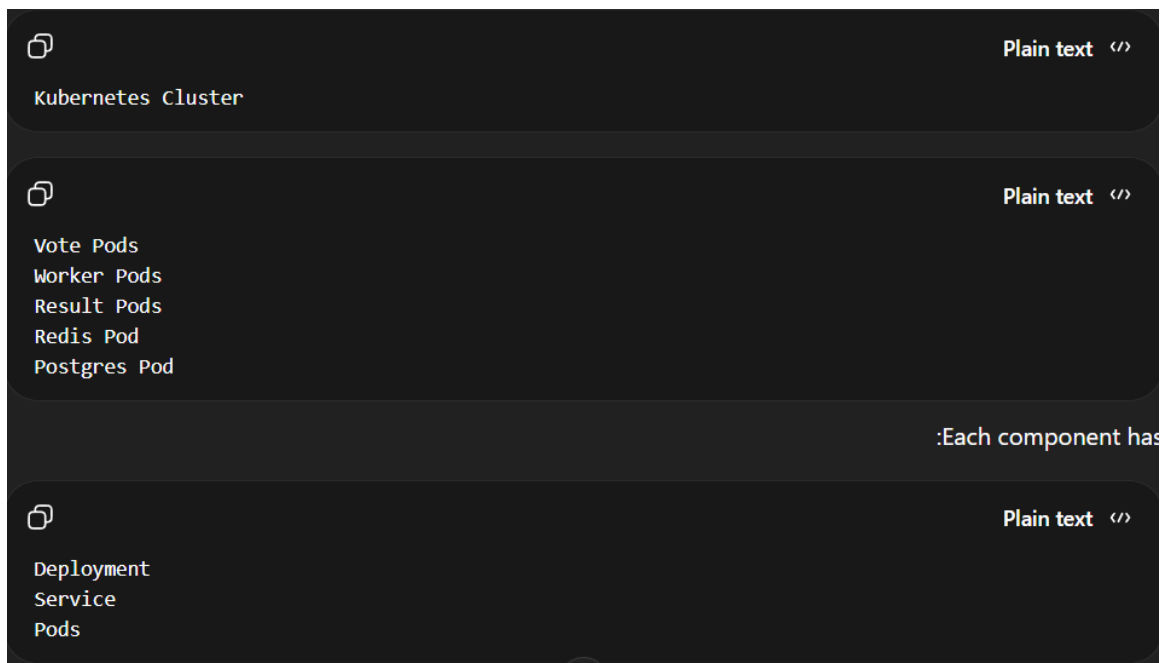
2 **2** Microservices Architecture Example (Voting App)



22.6 Microservices in Kubernetes

In Kubernetes, each microservice runs inside **its own Pods**.

Example cluster layout:



22.7 Service Communication in the Cluster

Microservices communicate using **Kubernetes Services**.

2 **2** Microservices Architecture Example (Voting App)

Example internal communication:



Plain text

```
Vote Service → Redis Service  
Worker Service → Postgres Service  
Result Service → Postgres Service
```

22.8 Example: Redis Service (ClusterIP)

Redis only needs to be accessed **inside the cluster**.

So we use **ClusterIP Service**.

apiVersion: v1

kind: Service



YAML

```
apiVersion: v1  
kind: Service  
metadata:  
  name: redis  
spec:  
  type: ClusterIP  
  selector:  
    app: redis  
  ports:  
    - port: 6379  
      targetPort: 6379
```

Communication:

Vote Service → redis:6379

Worker → redis:6379

22.9 Example: Postgres Service

2 **2** Microservices Architecture Example (Voting App)

```
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

Access inside the cluster:

Worker → db:5432

Result Service → db:5432

22.10 Example: Vote Service (NodePort)

Users must access the voting UI from outside the cluster.

So we expose it using **NodePort**.

```
apiVersion: v1
kind: Service
metadata:
  name: vote-service
spec:
  type: NodePort
  selector:
    app: vote
  ports:
    - port: 80
      targetPort: 80
      nodePort: 31000
```

Users access the application using:

```
http://NodeIP:31000
```

22.11 Example: Result Service

2 **2** Microservices Architecture Example (Voting App)

```
apiVersion: v1
kind: Service
metadata:
  name: result-service
spec:
  type: NodePort
  selector:
    app: result
  ports:
    - port: 80
      targetPort: 80
      nodePort: 31001
```

Users can view results using:

```
http://NodeIP:31001
```

22.12 Full Networking Flow

Complete traffic path:



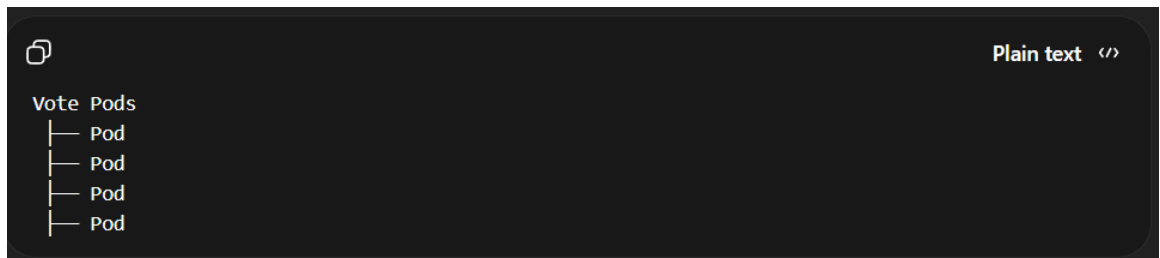
22.13 Why Microservices Work Well with Kubernetes

Kubernetes is designed to manage microservices because it provides:

2 **3** Deployments in Microservices Architecture

- Service discovery
- Load balancing
- Automatic scaling
- Self-healing Pods
- Independent deployments

Example scaling:



Only the vote service scales without affecting other services.

2 **3** Deployments in Microservices Architecture

23.1 Why Deployments are Used in Microservices

In Kubernetes, each microservice is typically managed using a **Deployment**.

A Deployment ensures:

- The desired number of Pods are running
- Pods are automatically recreated if they fail
- Applications can be updated safely using rolling updates
- Services remain available during updates

This is very important for **microservices**, because each service runs independently.

23.2 Microservices Deployment Structure

In the Voting App example, each service has its own Deployment.

Architecture:

2 **3** Deployments in Microservices Architecture

```

Kubernetes Cluster

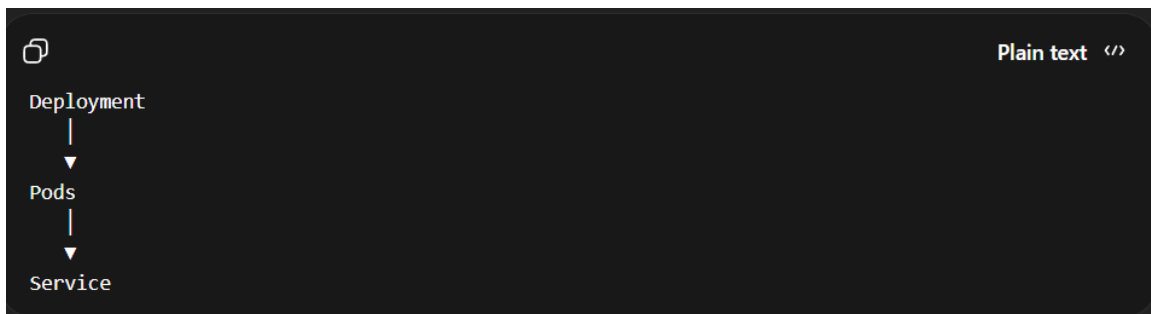
Vote Deployment
Worker Deployment
Result Deployment
Redis Deployment
Postgres Deployment

```

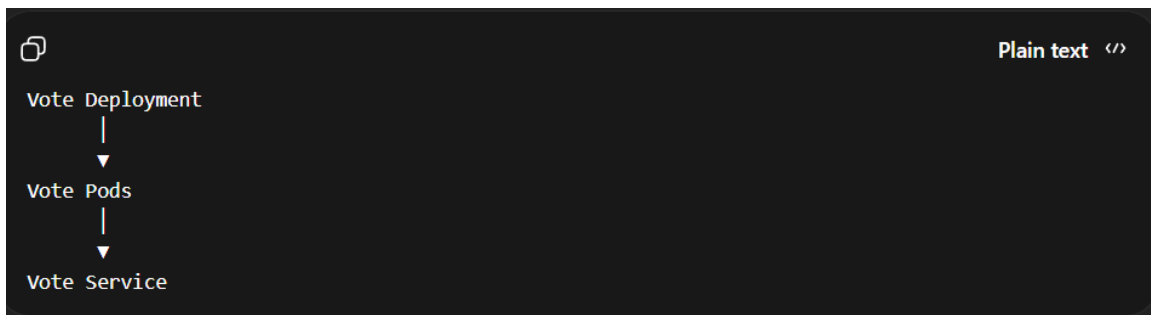
Each Deployment manages its own Pods.

23.3 Deployment and Service Relationship

Each microservice usually has:



Example:



The Service exposes the Pods managed by the Deployment.

23.4 Example: Vote Deployment

The Vote service is the frontend where users submit votes.

Example Deployment:

2 **3** Deployments in Microservices Architecture

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vote-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: vote
  template:
    metadata:
      labels:
        app: vote
    spec:
      containers:
        - name: vote
          image: kodekloud/examplevotingapp_vote
          ports:
            - containerPort: 80
```

Explanation:

- replicas: 3 → run three Pods
- selector → identifies Pods managed by this Deployment
- template → describes the Pod configuration

23.5 Example: Worker Deployment

The worker reads votes from Redis and writes them to Postgres.

Example Deployment:

2 **3** Deployments in Microservices Architecture

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: worker-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: worker
  template:
    metadata:
      labels:
        app: worker
    spec:
      containers:
        - name: worker
          image: kodekloud/examplevotingapp_worker
```

23.6 Example: Result Deployment

The result service displays the voting results.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: result-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: result
  template:
    metadata:
      labels:
        app: result
    spec:
      containers:
        - name: result
          image: kodekloud/examplevotingapp_result
          ports:
            - containerPort: 80
```

23.7 Example: Redis Deployment

Redis stores temporary vote data.

2 **3** Deployments in Microservices Architecture

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis
          ports:
            - containerPort: 6379
```

23.8 Example: Postgres Deployment

Postgres stores the final voting results.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres
          ports:
            - containerPort: 5432
```

23.9 Complete Microservices Deployment Diagram

2 **3** Deployments in Microservices Architecture

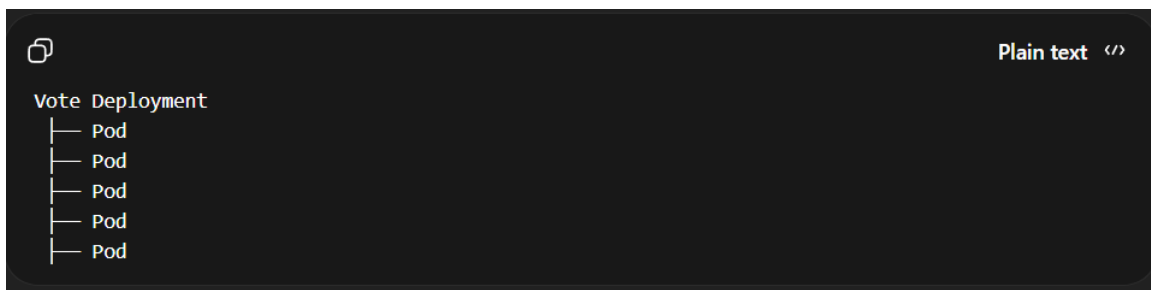


23.10 Scaling Microservices

One major advantage of microservices is **independent scaling**.

Example:

If many users are voting:



But Redis and Postgres may still run:

2 **3** Deployments in Microservices Architecture

 Plain text 

```
1 Pod each
```

This improves performance and resource usage.

23.11 Rolling Updates in Microservices

Deployments allow safe updates.

Example update:

 Plain text 

```
vote:v1 → vote:v2
```

Kubernetes performs a **rolling update**.

 Plain text 

```
Old Pods → gradually replaced → New Pods
```

The application remains available during the update.